

使用 Google Maps Platform Navigation SDK 建立簡單的 Android 導航應用程式

來源：<https://codelabs.developers.google.com/codelabs/maps-platform/navigation-sdk-101-android?hl=zh-tw>

步驟數：12

在本程式碼研究室中，您將學習如何使用 Google 地圖平台 Navigation SDK 建立簡單的導航應用程式。

目錄

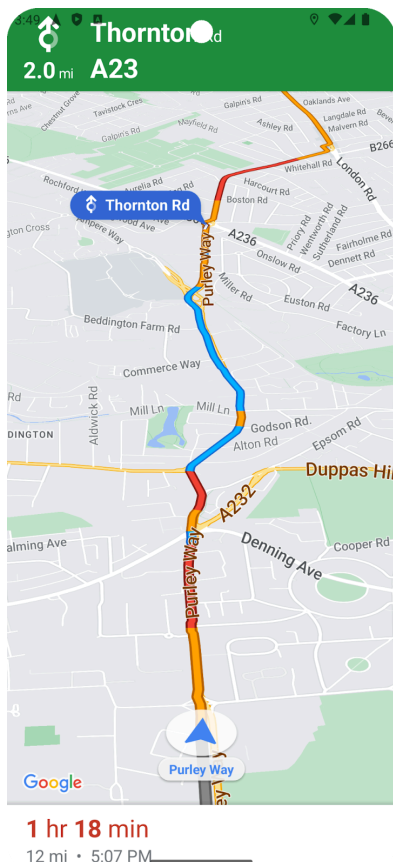
1. [1. 事前準備](#)
2. [2. 做好準備](#)
3. [3. 取得範例專案檔案](#)
4. [4. 在應用程式中加入 Navigation SDK](#)
5. [5. 設定應用程式權限並新增基本 UI](#)
6. [6. 新增導覽使用者介面](#)
7. [7. 初始化 Navigation SDK](#)
8. [8. 為重要導覽事件新增監聽器](#)
9. [9. 設定目的地](#)
10. [10. 建構及執行程式碼](#)
11. [11. 恭喜！](#)
12. [12. 後續步驟](#)

步驟 1 · 約 0 分鐘

1. 事前準備

本程式碼研究室將說明如何建立簡單的 Android 應用程式，使用 Google 地圖平台 Navigation SDK 導航至預先設定的目的地。

完成後，應用程式會如下所示。



必要條件

- 具備 Kotlin 的基本 Android 應用程式開發知識
- 對 Google 地圖 SDK 的基本概念 (例如地圖、位置、座標) 有初步瞭解。

課程內容

- 如何建立簡單的 Android 應用程式，使用 Navigation SDK 導航至目的地。
- 如何從遠端 Google Maven 存放區整合 Navigation SDK
- 如何管理位置存取權，以及使用者同意 Navigation SDK 使用者條款
- 如何初始化 SDK
- 如何設定目的地並開始導航指引。

軟硬體需求

- 已安裝最新穩定版 Android Studio。本程式碼研究室是使用 Android Studio Jellyfish 建立。如果您使用其他版本，介面和元件的外觀與版面配置可能會有所不同。
- 已啟用計費功能的 Google 帳戶和專案。

- 處於開發人員模式且已啟用 USB 偵錯功能的 Android 裝置，或是 Android 模擬器。無論選擇哪種方式，都必須符合 [Navigation SDK 的最低需求](#)

注意：Google Maps Platform 每月提供的 \$200 美元抵免額不適用於 Navigation SDK 用量。不過，每月前 1,000 個目的地不會產生費用。詳情請參閱「[用量與計費](#)」一文。

步驟 2 · 約 2 分鐘

2. 做好準備

如果您還沒有 Google Cloud Platform 帳戶，以及已啟用帳單的專案，請按照「開始使用 Google Maps Platform」的說明設定 Google Cloud 專案：<https://developers.google.com/maps/gmp-get-started>

在控制台中選取 Google Cloud 雲端專案

在 [Cloud 控制台](#) 中，按一下專案下拉式選單，然後選取要用於本程式碼研究室的專案。

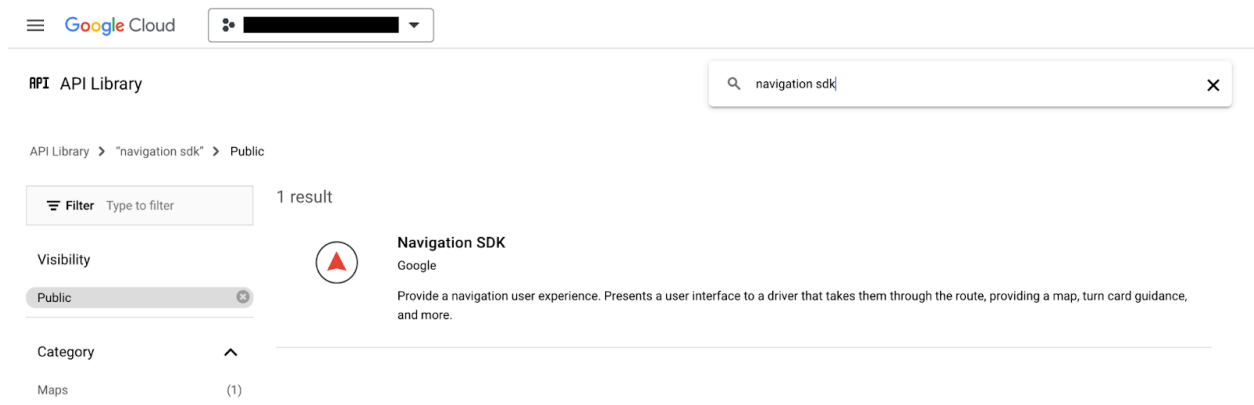
A blue header bar with a hamburger menu icon on the left, the text "Google Cloud Platform" in the center, and a "google.com" link with a dropdown arrow on the right.

在專案中啟用 Navigation SDK

在 [Google Cloud Marketplace](#) 中，啟用本程式碼研究室所需的 Google 地圖平台 API 和 SDK。

前往 Google Cloud 控制台的「API 和服務」>「程式庫」，然後搜尋「Navigation SDK」。

畫面應會顯示一項搜尋結果。



按一下 Navigation SDK 結果，開啟「產品詳細資料」頁面。按一下「啟用」按鈕，在專案中啟用 SDK。

針對 **Google Maps SDK for Android** 重複這個程序。

注意：Navigation SDK 內含 Google Maps Android SDK 版本，因此您不需要另外設定。如果將 Google 地圖 Android SDK 新增為個別依附元件，就會導致建構錯誤。

建立 API 金鑰

在 Cloud Console 的「憑證」頁面中產生 API 金鑰。您可以按照「[開始使用 Google 地圖平台](#)」一文快速入門部分的步驟 3 操作。所有 Google Maps Platform 要求都需要 API 金鑰。

3. 取得範例專案檔案

本節說明如何從本程式碼研究室的 GitHub 存放區複製檔案，設定基本的空白 Android Studio 專案。Github 存放區包含程式碼研究室程式碼的先前和後續版本。本程式碼研究室會從空白專案範本開始，逐步建構至完成狀態。如果遇到問題，可以參考存放區中的完成專案。

複製這個 GitHub 存放區，即可取得本程式碼研究室的程式碼。

```
git clone https://github.com/googlemaps-samples/codelab-navigation-101-android-kotlin.git
```

如果沒有安裝 Git，請點選這個按鈕取得程式碼：

為協助您盡快上手，存放區的 `starter` 資料夾中提供了一些範例程式碼，方便您跟著本程式碼研究室的說明操作。入門專案提供基本的應用程式 UI 和建構設定，但未加入 Navigation SDK。此外，我們也提供已完成的 `solution` 專案，方便您隨時跳過或查看進度。

在 Android Studio 中開啟複製的存放區

在本機複製存放區後，請使用 Android Studio 開啟 `starter` 資料夾，做為現有專案。

1. 在「Welcome to Android Studio」對話方塊中，按一下「Open」按鈕。
2. 前往儲存已複製存放區的資料夾，然後選取頂層「`codelab-navigation-101-android-kotlin`」資料夾內的「`starter`」資料夾。
3. 確認專案是否能建構及執行。

注意：本程式碼研究室的專案是以 Kotlin 編寫。

新增虛擬裝置或連結硬體裝置

如要將 Android 裝置連線至電腦，請按照 Android Studio 的「[在硬體裝置上執行應用程式](#)」一文說明操作。您也可以使用 [Android 虛擬裝置管理工具 \(AVD Manager\)](#) 設定虛擬裝置。選擇模擬器時，請務必挑選包含 Google API 的映像檔。

在 Android Studio 中，按一下「Run」(執行) 選單選項或播放按鈕圖示，然後按照系統提示選擇裝置。

4. 在應用程式中加入 Navigation SDK

將 Navigation SDK 程式庫和 API 金鑰新增至專案

注意：存放區中的專案會使用 Kotlin DSL 建立建構檔案，而 [Navigation SDK 示範應用程式](#) 則會使用 Groovy DSL。語法已遷移，但您可能會發現 Kotlin 指定項目的方式略有不同。請參閱[遷移指南](#)，瞭解兩者的差異。

如要將 Navigation SDK 程式庫新增至應用程式，您需要修改應用程式層級的 `build.gradle.kts`，從 Google Maven 存放區擷取 Navigation SDK，並設定版本號碼。

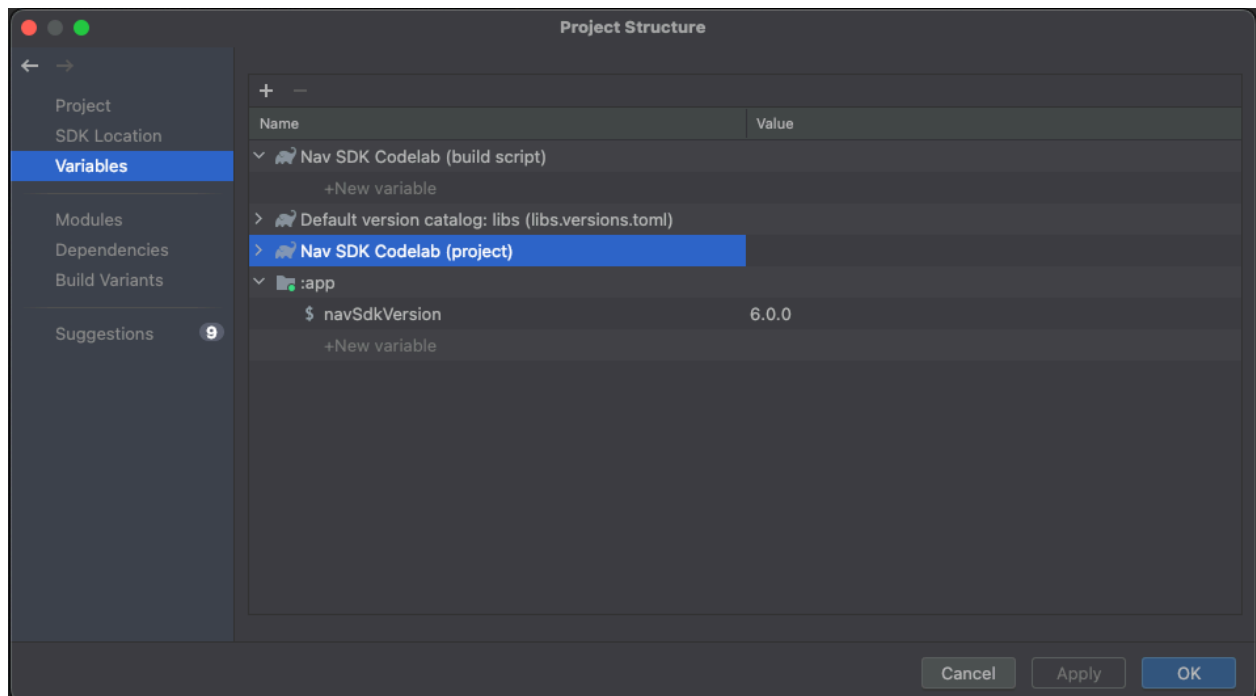
在建構設定中建立變數，用於儲存 Navigation SDK 版本號碼。

在應用程式層級的 `build.gradle.kts` 中設定變數，內含應用程式使用的 Navigation SDK 版本值，方便日後改用最新版本。

如要查看最新版本號碼，請參閱「[Navigation SDK 版本資訊](#)」。

```
val navSdkVersion by extra("6.0.0")
```

您也可以使用「File」>「Project Structure」>「Variables」中的對話方塊，修改這個和其他變數的值：



在建構設定中新增依附元件

現在，請在應用程式層級 `build.gradle.kts` 的依附元件區塊中加入下列 API 依附元件。使用的版本會是您剛在應用程式層級 `build.gradle.kts` 中設定的 `${navSdkVersion}` 值：

```
dependencies {  
  
    // Include the Google Navigation SDK.  
    api("com.google.android.libraries.navigation:navigation:${navSdkVersion}")  
  
    ...  
}
```

新增 API 金鑰

使用 Secrets Gradle 外掛程式管理 API 金鑰

建議您使用 [Secrets Gradle 外掛程式](#)，在應用程式中安全地管理 API 金鑰。外掛程式已在頂層 `build.gradle.kts` 檔案中，以依附元件的形式新增至初始專案範本。

```
// Top-level build file where you can add configuration options common to all sub-  
projects/modules.  
plugins {  
    id("com.google.android.libraries.mapsplatform.secrets-gradle-plugin") version "2.0.1"  
    apply false  
    //... other plugin definitions here  
}
```

開啟頂層目錄中的 `secrets.properties` 檔案，然後將 `YOUR_API_KEY` 換成您的 API 金鑰。`secrets.properties` 不會登錄在版本管控系統中，因此請將金鑰儲存至該檔案。

```
MAPS_API_KEY=YOUR_API_KEY
```

若要進一步瞭解這個主題，請參閱 Navigation SDK 說明文件中的「[將 API 金鑰加到應用程式](#)」。

驗證 local.defaults.properties 的內容

空白專案的頂層目錄中也有 `local.defaults.properties` 檔案，與 `secrets.properties` 檔案位於同一資料夾。開啟該檔案，並查看下列程式碼。

```
MAPS_API_KEY=DEFAULT_API_KEY
```

如果專案未新增 `secrets.properties`，這個檔案便可做為 `MAPS_API_KEY` 屬性的備份值，以確保建置程序不會失敗。您不需要編輯這個檔案。如果找不到 `MAPS_API_KEY` 的 `secrets.properties` 定義，預設值會停止在執行階段執行應用程式，並顯示 API 金鑰錯誤。

確認 Android 資訊清單使用您指定的 API 金鑰

開啟 `app/src/main/AndroidManifest.xml`。您會發現 `MAPS_API_KEY` 屬性是用來設定應用程式的 API 金鑰：

```
<meta-data  
    android:name="com.google.android.geo.API_KEY"  
    android:value="{MAPS_API_KEY}" />
```

開啟應用程式層級的 `build.gradle.kts` 檔案，然後找出 `secrets` 屬性。

外掛程式的 `propertiesFileName` 設定應設為 `secrets.properties`，且 `defaultPropertiesFileName` 應讀取 `local.defaults.properties`。

```
secrets {  
    // Optionally specify a different file name containing your secrets.  
    // The plugin defaults to "local.properties"  
    propertiesFileName = "secrets.properties"  
  
    // A properties file containing default secret values. This file can be  
    // checked in version control.  
    defaultPropertiesFileName = "local.defaults.properties"  
}
```

儲存所有檔案，並使用 Gradle 同步處理專案。

5. 設定應用程式權限並新增基本 UI

要求精確位置存取權

Navigation SDK 必須依賴 GPS 訊號才能運作，因此應用程式需要要求使用者授予精確位置資料的存取權。將存取精確位置的權限新增為 AndroidManifest.xml 中 `<manifest>` 元素的子項。

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools" >
    <uses-permission
        android:name="android.permission.ACCESS_FINE_LOCATION"
    />
</manifest>
```

如要進一步瞭解 Android 位置存取權，請參閱 Android 開發人員說明文件的「[要求位置存取權](#)」一節。

如要在 Android 14 裝置上執行應用程式，請在與精確位置資訊存取權相同的位置新增下列 `uses-permission` 標記，要求前景服務位置存取權：

```
<uses-permission android:name="android.permission.FOREGROUND_SERVICE_LOCATION" />
```

新增具有基本 UI 的啟動活動

應用程式執行時，需要啟動期間執行的程式碼，檢查使用者是否已授予位置存取權，並處理各種可能情況，如果尚未授予權限，則要求權限。如要這麼做，請在應用程式中新增基本使用者介面。本程式碼研究室會使用您在 Android Studio 中建立新的空白 Views 活動時建立的 UI。您將調整這項作業，在將程式碼新增至活動的導覽 UI 前，先執行位置存取權檢查。

在程式碼編輯器中開啟 `MainActivity.kt` 檔案，並檢查程式碼，其中會顯示基本 UI。

在執行階段要求位置存取權

應用程式必須在初始化 Navigation SDK 之前，觸發精確位置資訊存取權要求。

如要確保應用程式啟動時會進行這項檢查，請在 Activity 的覆寫 `onCreate()` 方法中，將一些程式碼新增至 `MainActivity` 類別。

下列程式碼會檢查使用者是否已授予精確位置存取權；如果未授予，程式碼就會要求權限。在 `onCreate()` 方法中新增這段程式碼。

```

val permissions =
    if (VERSION.SDK_INT >= VERSION_CODES.TIRAMISU) {
        arrayOf(permission.ACCESS_FINE_LOCATION, permission.POST_NOTIFICATIONS)
    } else {
        arrayOf(permission.ACCESS_FINE_LOCATION)
    }

if (permissions.any { !checkPermissionGranted(it) }) {

    if (permissions.any { shouldShowRequestPermissionRationale(it) }) {
        // Display a dialogue explaining the required permissions.
    }

    val permissionsLauncher =
        registerForActivityResult(
            RequestMultiplePermissions(),
            { permissionResults ->
                if (permissionResults.getDefault(permission.ACCESS_FINE_LOCATION, false)) {
                    onLocationPermissionGranted()
                } else {
                    finish()
                }
            },
        )

    permissionsLauncher.launch(permissions)
} else {
    android.os.Handler(Looper.getMainLooper()).postDelayed({ onLocationPermissionGranted()
}, SPLASH_SCREEN_DELAY_MILLIS)
}

private fun checkPermissionGranted(permissionToCheck: String): Boolean =
    ContextCompat.checkSelfPermission(this, permissionToCheck) ==
    PackageManager.PERMISSION_GRANTED

```

在 `MainActivity` 類別中新增名為 `onLocationPermissionGranted` 的函式，處理使用者授予位置資訊分享權限時的結果。在接下來的步驟中，我們將在此新增程式碼，啟動新的導覽活動。

```

private fun onLocationPermissionGranted() {
    //code to initialize Navigation SDK will go here
}

```

建構專案。如有任何建構錯誤，請找出並修正。

在新虛擬裝置上執行專案。應用程式安裝並啟動時，您應該會看到權限要求對話方塊。

6. 新增導覽使用者介面

新增 Navigation UI 的方式有兩種：`SupportNavigationFragment` 或 `NavigationView`。

為求簡化，本程式碼研究室使用 `NavigationView`。

編輯版面配置

編輯 `res/layout/activity_main.xml`，為 `NavigationView` 新增版面配置。

1. 開啟檔案並切換至程式碼檢視模式。
2. 將檔案的全部內容替換成 `RelativeLayout` 內的 `NavigationView` 新版面配置，如下例所示。您只需要在應用程式中新增導覽檢視區塊，因此簡單的版面配置就足夠了。
3. 將 `NavigationView` 的 ID 設為「`@+id/navigation_view`」。

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <com.google.android.libraries.navigation.NavigationView
        android:id="@+id/navigation_view"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        />
</RelativeLayout>
```

設定 Navigation 活動

在 Android Studio 中，於編輯器開啟 `MainActivity.kt` 檔案。

加入一些基本設定程式碼，確保應用程式中的導覽體驗正常運作。在 `MainActivity.kt` 檔案中，進行下列變更：

1. 在 `MainActivity` 類別中宣告變數，以參照 `NavigationView`：

```
private lateinit var navView: NavigationView
```

2. 在 `onCreate()` 方法中新增一些程式碼，取得 `NavigationView` 的參照：

```
navView = findViewById(R.id.navigation_view)
navView.onCreate(savedInstanceState)
```

3. 在 `onCreate()` 方法中新增一些程式碼，確保導航指引期間螢幕保持開啟狀態：

```
// Ensure the screen stays on during nav.
window.addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)
```

4. 編輯呼叫 `ViewCompat.setOnApplyWindowInsetsListener` 的程式碼，以參照 `NavigationView` 的 ID。

```
ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.navigation_view)) { v, insets ->
    val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
    v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
    insets
}
```

5. 在類別中新增 `showToast()` 方法，向使用者顯示意見回饋：

```
private fun showToast(errorMessage: String) {
    Toast.makeText(this@MainActivity, errorMessage, Toast.LENGTH_LONG).show()
}
```

7. 初始化 Navigation SDK

您已完成基本的 Navigation 活動設定，現在可以初始化 Navigation SDK。如要這麼做，請在 MainActivity.kt 檔案中加入下列程式碼：

```
/** Starts the Navigation API, capturing a reference when ready. */
@SuppressLint("MissingPermission")
private fun initializeNavigationApi() {
    NavigationApi.getNavigator(
        this,
        object : NavigatorListener {
            override fun onNavigatorReady(navigator: Navigator) {
                // store a reference to the Navigator object
                mNavigator = navigator
                // code to start guidance will go here
            }

            override fun onError(@NavigationApi.ErrorCode errorCode: Int) {
                when (errorCode) {
                    NavigationApi.ErrorCode.NOT_AUTHORIZED -> {
                        // Note: If this message is displayed, you may need to check that
                        // your API_KEY is specified correctly in AndroidManifest.xml
                        // and is been enabled to access the Navigation API
                        showToast(
                            "Error loading Navigation API: Your API key is " +
                                "invalid or not authorized to use Navigation."
                        )
                    }
                    NavigationApi.ErrorCode.TERMS_NOT_ACCEPTED -> {
                        showToast(
                            "Error loading Navigation API: User did not " +
                                "accept the Navigation Terms of Use."
                        )
                    }
                    else -> showToast("Error loading Navigation API: $errorCode")
                }
            }
        },
    )
}
```

這段程式碼會建立名為 `initializeNavigationApi()` 的新方法。這個方法會呼叫 `NavigationApi.getNavigator()` 取得 `Navigator` 物件的參照，並實作 `NavigatorListener` 來處理回呼。

請注意，初始化 Navigation API 時，系統會叫用 `NavigatorListener.onNavigatorReady` 方法，並將 `Navigator` 物件做為參數傳遞。上述程式碼會使用傳遞至這個方法的已初始化 `Navigator` 物件，更新您先前宣告的 `mNavigator` 變數。

注意：如果呼叫的 API 方法需要使用者授予位置存取權，Android Studio 會發出權限遺漏警告，但 `@SuppressWarnings` 註解可避免這種情況。

最後，從 `onLocationPermissionGranted` 方法新增對 `initializeNavigationApi` 方法的呼叫。

```
private fun onLocationPermissionGranted() {  
    initializeNavigationApi()  
}
```

8. 為重要導覽事件新增監聽器

使用者遵循導覽指示時，Navigation SDK 會觸發事件，通知應用程式路途中的重要狀態變化，例如使用者重新規劃路線或抵達目的地。在 MainActivity.kt 檔案中，新增監聽器來處理這些事件：

1. 在 MainActivity 類別中，宣告兩個變數來參照事件監聽器物件：

```
private var arrivalListener: Navigator.ArrivalListener? = null
private var routeChangeListener: Navigator.RouteChangeListener? = null
```

2. 初始化 Navigator 時，請新增 registerNavigationListeners() 方法來設定監聽器。當系統觸發 Arrival 事件時，這個方法會呼叫 Navigator.clearDestinations() 來重設 NavigationView：

```
/**
 * Registers a number of example event listeners that show an on screen message when certain
 * navigation events occur (e.g. the driver's route changes or the destination is reached).
 */
private fun registerNavigationListeners() {
    withNavigatorAsync {
        arrivalListener =
            Navigator.ArrivalListener { // Show an onscreen message
                showToast("User has arrived at the destination!")
                mNavigator?.clearDestinations()
            }
        mNavigator?.addArrivalListener(arrivalListener)

        routeChangeListener =
            Navigator.RouteChangeListener { // Show an onscreen message when the route
changes
                showToast("onRouteChanged: the driver's route changed")
            }
        mNavigator?.addRouteChangeListener(routeChangeListener)
    }
}
```

3. 在 initializeNavigationApi 方法中，從 onNavigatorReady 回呼程式碼呼叫 registerNavigationListeners()：

```
override fun onNavigatorReady(navigator: Navigator) {
    // store a reference to the Navigator object
    mNavigator = navigator

    //listen for events en route
    registerNavigationListeners()
}
```


4. 設定使用者介面。在導覽期間，您可以控管導覽使用者介面的各個層面。其中一項重要的自訂項目是攝影機位置。在 `onNavigatorReady` 中，對 `navigator` 物件傳回的 `setTaskRemovedBehaviour` 方法新增呼叫，如下所示。如果滑開應用程式，系統會終止指引和通知：

```
// Disables the guidance notifications and shuts down the app and background service
// when the user dismisses/swipes away the app from Android's recent tasks.
navigator.setTaskRemovedBehavior(Navigator.TaskRemovedBehavior.QUIT_SERVICE)
```

5. 新增對 `GoogleMap.followMyLocation` 的呼叫，以指定 `CameraPerspective`。 `GoogleMap` 可透過 `NavigatorView.getMapAsync()` 方法存取，如下所示：

```
navView.getMapAsync {
    googleMap ->
    googleMap.followMyLocation(GoogleMap.CameraPerspective.TILTED)
}
```

6. 為確保導覽功能在整個應用程式生命週期中順暢運作，請在 `MainActivity` 類別中實作下列方法：

```
override fun onSaveInstanceState(savedInstanceState: Bundle) {
    super.onSaveInstanceState(savedInstanceState)

    navView.onSaveInstanceState(savedInstanceState)
}

override fun onTrimMemory(level: Int) {
    super.onTrimMemory(level)
    navView.onTrimMemory(level)
}

override fun onStart() {
    super.onStart()
    navView.onStart()
}

override fun onResume() {
    super.onResume()
    navView.onResume()
}

override fun onPause() {
    navView.onPause()
    super.onPause()
}

override fun onConfigurationChanged(configuration: Configuration) {
    super.onConfigurationChanged(configuration)
    navView.onConfigurationChanged(configuration)
}

override fun onStop() {
    navView.onStop()
    super.onStop()
}

override fun onDestroy() {
    navView.onDestroy()
    withNavigatorAsync {
        // Unregister event listeners to avoid memory leaks.
        if (arrivalListener != null) {
            navigator.removeArrivalListener(arrivalListener)
        }
        if (routeChangeListener != null) {
            navigator.removeRouteChangeListener(routeChangeListener)
        }

        navigator.simulator?.unsetUserLocation()
        navigator.cleanup()
    }
}
```

```
super.onDestroy()
```

```
}
```

9. 設定目的地

現在可以設定目的地，並開始導覽指引。在 `MainActivity.kt` 檔案中，進行下列變更：

1. 新增 `navigateToPlace()` 方法，設定導覽目的地並接受 `placeId` 參數。

```
/**
 * Requests directions from the user's current location to a specific place (provided by the
 * Place ID).
 */
private fun navigateToPlace(placeId: String) {

}
```

2. 在 `navigateToPlace()` 方法中，使用 `Waypoint.builder()` 方法從傳遞至該方法的地點 ID 建立 `Waypoint`。處理這項作業可能會擲回的 `UnsupportedPlaceIdException`，適用於地點 ID 無法解析為精確地址的情況：

```
val waypoint: Waypoint? =
// Set a destination by using a Place ID (the recommended method)
try {
    Waypoint.builder().setPlaceIdString(placeId).build()
} catch (e: Waypoint.UnsupportedPlaceIdException) {
    showToast("Place ID was unsupported.")
    return
}
```

3. 在 `navigateToPlace()` 方法中加入下列程式碼，使用 `Waypoint` 設定目的地：

```
val pendingRoute = mNavigator?.setDestination(waypoint)

// Set an action to perform when a route is determined to the destination
pendingRoute?.setOnResultListener { code ->
    when (code) {
        RouteStatus.OK -> {
            // Code to start guidance will go here
        }

        RouteStatus.ROUTE_CANCELED -> showToast("Route guidance canceled.")
        RouteStatus.NO_ROUTE_FOUND,
        RouteStatus.NETWORK_ERROR ->
            // TODO: Add logic to handle when a route could not be determined
            showToast("Error starting guidance: $code")

        else -> showToast("Error starting guidance: $code")
    }
}
```

`Navigator` 物件具有 `setDestinations()` 方法，可接受各種參數。最基本的做法是提供 `Waypoint`。系統預設會使用 `DRIVING` 交通方式，適合四輪車輛。`setDestinations()` 方法會傳回包含 `RouteStatus` 物件的

`ListenableResultFuture` 物件。`RouteStatus` 會指出是否已找到前往目的地的路線，如果沒有，您也可以處理各種錯誤狀態。

4. 進行其他設定變更，提升導覽使用者體驗：

```
// Hide the toolbar to maximize the navigation UI
supportActionBar?.hide()

// Enable voice audio guidance (through the device speaker)
mNavigator?.setAudioGuidance(Navigator.AudioGuidance.VOICE_ALERTS_AND_GUIDANCE)

// Simulate vehicle progress along the route (for demo/debug builds)
if (BuildConfig.DEBUG) {
    mNavigator?.simulator?.simulateLocationsAlongExistingRoute(
        SimulationOptions().speedMultiplier(5f)
    )
}
```

包括：

- 隱藏動作列，盡量騰出空間給導覽 UI。
- 啟用語音導引，讓系統朗讀警報和導航指示。
- 指定速度乘數，設定模擬器以進行偵錯。

5. 找出要做為目的地的地點 ID。最好不要離使用者位置太遠。使用 [Google Maps Platform 地點 ID 搜尋器公用程式](#)，或透過 Places API 呼叫取得地點 ID。

如果您要模擬導航，可以在程式碼中設定使用者位置，也可以從連結裝置取得位置。本程式碼研究室會假設您在模擬英國倫敦的位置。

6. 在 `MainActivity` 類別中新增伴隨物件，儲存起始位置和地點 ID。本程式碼研究室會使用倫敦的起始位置和特拉法加廣場的地點 ID：

```
companion object{
    const val TRAFALGAR_SQUARE = "ChIJH-tB0c4EdkgRJ8aJ8P1CUxo" //London, UK
    val startLocation = LatLng(51.345678, -0.1234456)
}
```

提示：如要取得其他經緯度做為起點，請在瀏覽器中於 Google 地圖上按一下滑鼠右鍵，或在 Google 地圖行動應用程式中長按地圖。

7. 在 `initializeNavigationApi` 方法內的 `onNavigatorReady` 回呼中，新增對 `navigateToPlace()` 方法的呼叫，並新增將在偵錯模式下執行的邏輯分支，以設定使用者位置：

```
// Disables the guidance notifications and shuts down the app and background service
// when the user dismisses/swipes away the app from Android's recent tasks.
navigator.setTaskRemovedBehavior(Navigator.TaskRemovedBehavior.QUIT_SERVICE)

mNavigator = navigator

if (BuildConfig.DEBUG) {
    mNavigator?.simulator?.setUserLocation(MainActivity.startLocation)
}
//listen for events en route
registerNavigationListeners()

navView.getMapAsync {
    googleMap ->
    googleMap.followMyLocation(GoogleMap.CameraPerspective.TILTED)
}

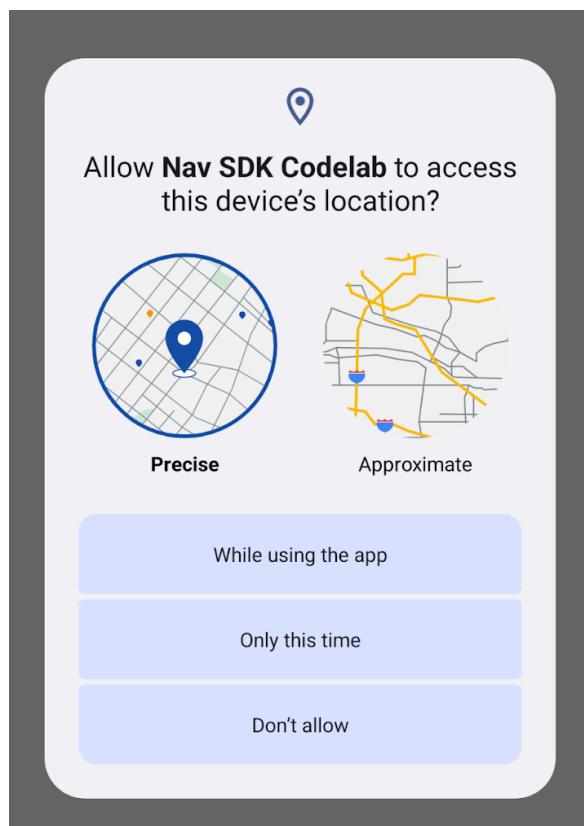
//navigate to a destination
navigateToPlace(MainActivity.TRAFALGAR_SQUARE)
```

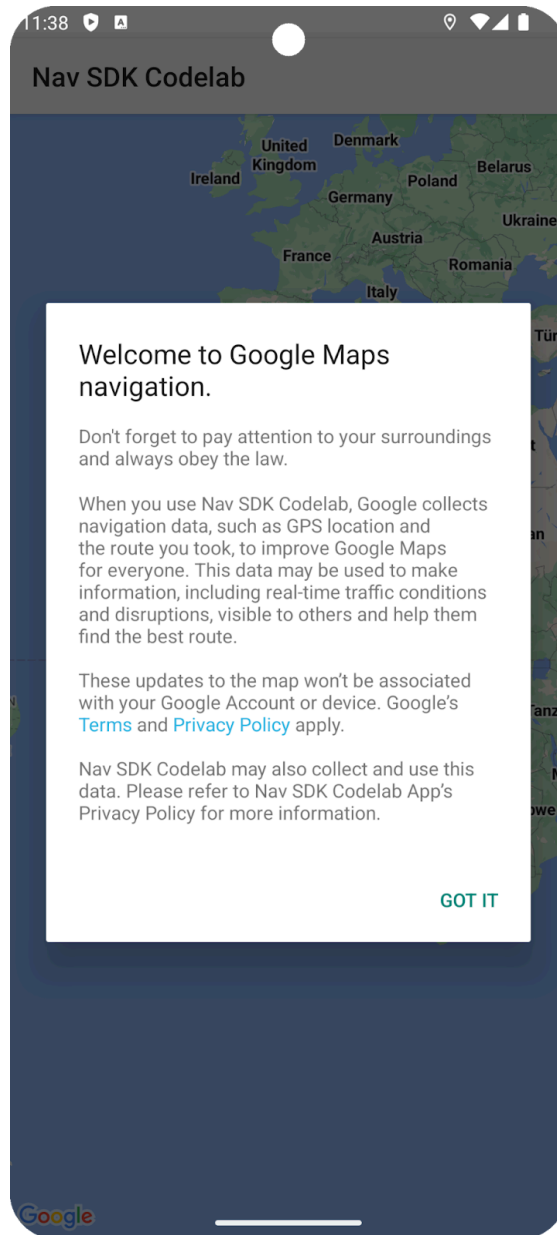
步驟 10 · 約 0 分鐘

10. 建構及執行程式碼

首次執行應用程式時，您需要授予應用程式位置存取權，並接受 Navigation SDK 使用條款。

注意：執行應用程式會呼叫 `setDestinations()` 方法，使用前 1000 個目的地後，就會開始收費。詳情請參閱「[用量與計費](#)」一文。

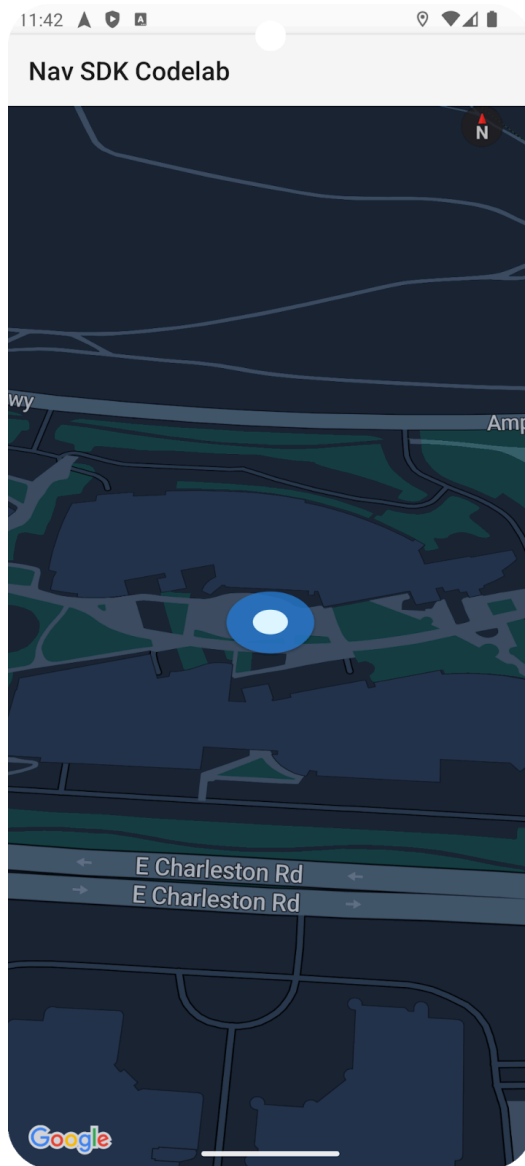




設定位置

除非您已在程式碼中設定位置，或使用模擬器屬性對話方塊，否則模擬裝置的位置預設會設為加州山景城的 Google 園區。

如果發生這種情況，您可能會發現應用程式找不到您設定的地點 ID 路線 (預設為澳洲雪梨雪梨歌劇院)。 `showToast()` 方法會顯示「找不到路線」訊息。



將起始位置硬式編碼

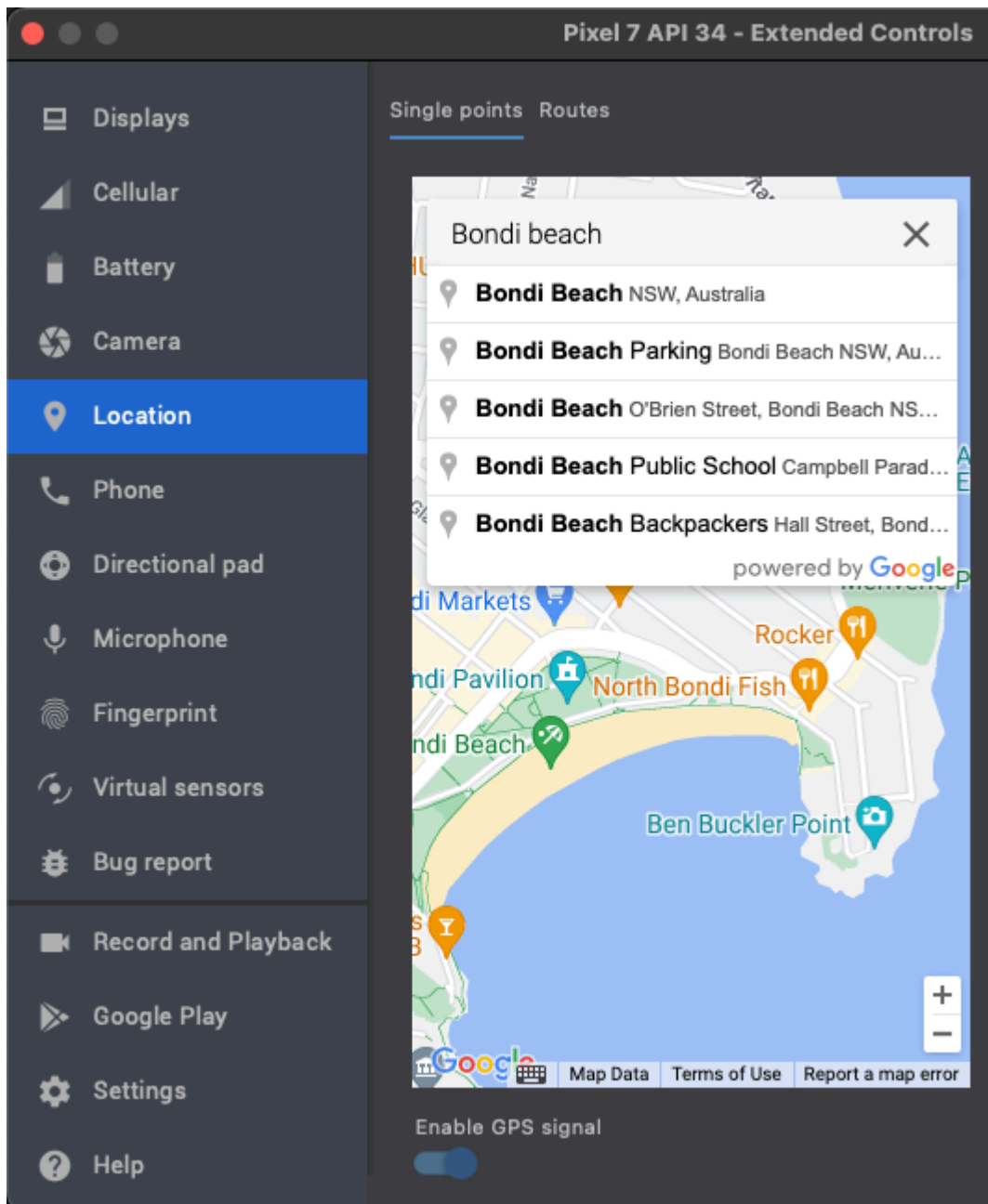
如要在程式碼中設定其他位置，請在 MainActivity.kt 的 `navigateToPlace()` 方法中，於呼叫 `mNavigator.startGuidance()` 前新增下列程式碼：

```
mNavigator?.simulator?.setUserLocation(startLocation)
```

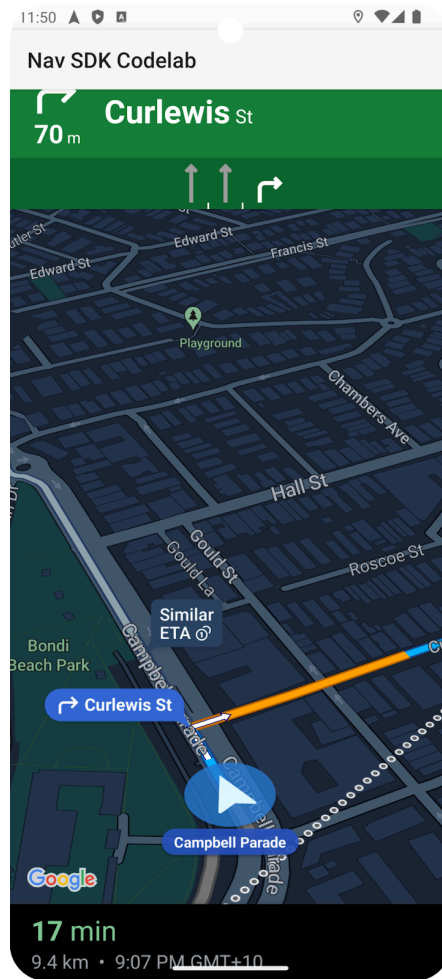
在您選擇的預設位置啟動模擬器

如要在裝置模擬器中設定其他位置，請啟動模擬器 (如果尚未啟動)，然後按一下 3 點選單，工具提示會顯示「Extended Controls」(擴充控制項)。開啟的對話方塊會顯示「位置」選單選項。

舉例來說，如果您使用雪梨歌劇院的地點 ID 做為目的地，請選擇澳洲雪梨的位置。舉例來說，搜尋「邦迪海灘」，選取建議，然後按一下對話方塊右下方的「儲存地點」。你也可以按一下「儲存點」，將地點新增至儲存清單，以供日後使用。



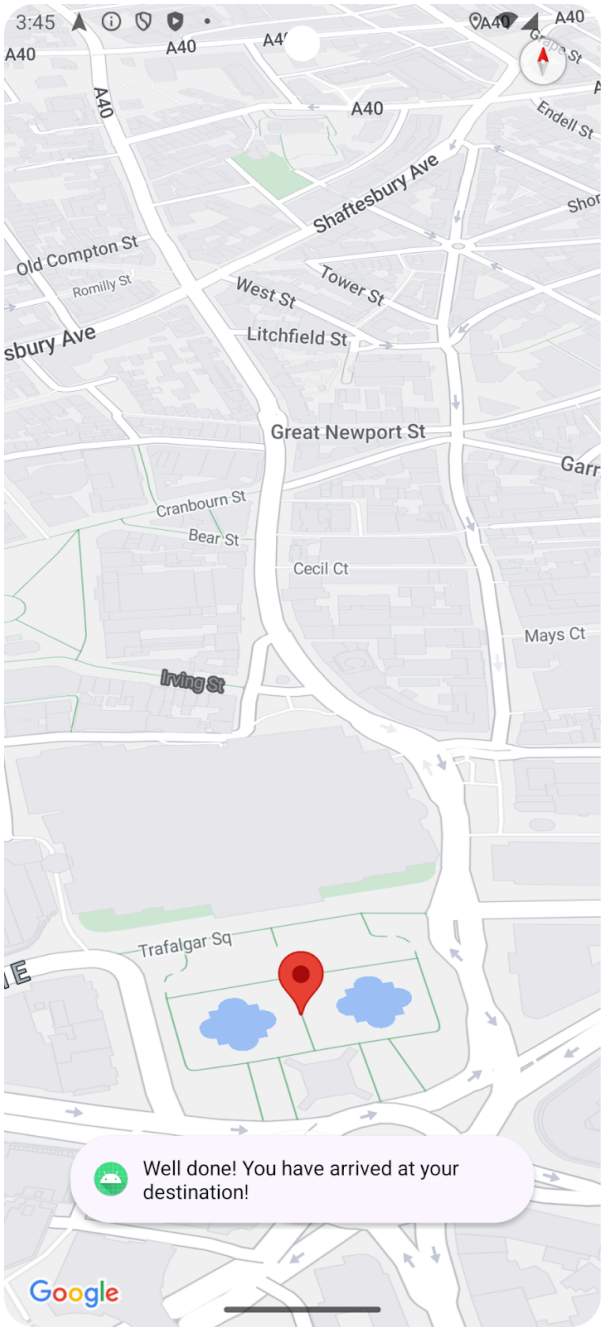
如果將其他地點 ID 設為目的地，請選擇附近的地點，確保模擬路線符合實際情況，且不會太長，方便進行偵錯。
重新啟動應用程式，現在應該會導覽至目的地。



步驟 11 · 約 0 分鐘

11. 恭喜！

您已完成本程式碼研究室。恭喜！你已抵達目的地。祝您編寫程式一切順利 :-)



12. 後續步驟

如要進一步開發應用程式，請參考下列主題，尋找靈感。

- [監聽更多導覽事件](#)。新增程式碼，在司機重新規劃路線或抵達時顯示訊息。
 - [自訂導覽介面](#)。
 - 如果想挑戰更高難度的任務，不妨新增 Places API 地點挑選器控制項，讓使用者設定目的地。提示：[GitHub 上的 Navigation SDK 試用版應用程式](#)提供實作範例。
 - 如要避免非同步呼叫 Navigator 和 GoogleMap 物件時發生潛在問題，請採用 [Github 上的 Navigation SDK 試用版應用程式](#)所用的方法。在較複雜的應用程式情境中，程式碼執行時，這些物件可能尚未完成初始化。提示：您可以在 MainActivity.kt 檔案結尾加入 [InitializedNavScope](#) 類別，快速完成實作。
-